Payments engine CSV transactions

Transcript: transcript_exercise_A.jsonl
Directory: /Users/mathieudutoursikiric/GITall/Kraken/Take_home/exercise
Session: 5c5915d7-8204-49f7-bd44-7dd9b3dce5de
Date: 2026-10-18 11:26 → 2026-10-18 12:55

Prompt 1 2026-10-18 11:26

```
Given a CSV representing a series of transactions, implement a simple toy
↪  transactions engine
in rust that processes the payments crediting and debiting accounts. After
↪  processing the
complete set of payments output the client account balances

You should be able to run your payments engine like
$ cargo run -- transactions.csv > accounts.csv
The input file is the first and only argument to the binary. Output should be
↪  written to std out

* Input *

The input will be a CSV file with the columns type, client, tx, and amount. You can
↪  assume the
type is a string, the client column is a valid u16 client ID, the tx is a valid u32
↪  transaction ID, and
the amount is a decimal value with a precision of up to four places past the
↪  decimal.
For example
type, client, tx, amount
deposit, 1, 1, 1.0
deposit, 2, 2, 2.0
deposit, 1, 3, 2.0
withdrawal, 1, 4, 1.5
withdrawal, 2, 5, 3.0
The client ID will be unique per client though are not guaranteed to be ordered.
↪  Transactions to
the client account 2 could occur before transactions to the client account 1.
↪  Likewise, transaction
IDs (tx) are globally unique, though are also not guaranteed to be ordered. You can
↪  assume the
transactions occur chronologically in the file, so if transaction b appears after a
↪  in the input file
then you can assume b occurred chronologically after a. Whitespaces and decimal
↪  precisions
(up to four places past the decimal) must be accepted by your program.

* Output *
The output should be a list of client IDs (client), available amounts (available),
↪  held amounts
(held), total amounts (total), and whether the account is locked (locked). Columns
↪  are defined as
Column Description
available The total funds that are available for trading, staking, withdrawal, etc.
This should be equal to the total - held amounts
held The total funds that are held for dispute. This should be equal to
total - available amounts
total The total funds that are available or held. This should be equal to available
+ held
```

```
locked Whether the account is locked. An account is locked if a charge back occurs
For example
client, available, held, total, locked
1, 1.5, 0.0, 1.5, false
2, 2.0, 0.0, 2.0, false
Spacing and displaying decimals for round values do not matter. Row ordering also
↪  does not
matter. The above output will be considered the exact same as the following
client,available,held,total,locked
2,2,0,2,false
1,1.5,0,1.5,false
Precision
You can assume a precision of four places past the decimal and should output values
↪  with the
same level of precision.
Types of Transactions

** Deposit **
A deposit is a credit to the client's asset account, meaning it should increase the
↪  available and
total funds of the client account
A deposit looks like
   type    client tx amount
   deposit 1      1  1.0

** Withdrawal **
A withdraw is a debit to the client's asset account, meaning it should decrease the
↪  available and
total funds of the client account
A withdrawal looks like
   type       client tx amount
   withdrawal 2      2  1.0
If a client does not have sufficient available funds the withdrawal should fail and
↪  the total amount
of funds should not change

** Dispute **
A dispute represents a client's claim that a transaction was erroneous and should
↪  be reversed.
The transaction shouldn't be reversed yet but the associated funds should be held.
↪  This means
that the clients available funds should decrease by the amount disputed, their held
↪  funds should
increase by the amount disputed, while their total funds should remain the same.
A dispute looks like
   type     client tx amount
   dispute  1      1
Notice that a dispute does not state the amount disputed. Instead a dispute
↪  references the
transaction that is disputed by ID. If the tx specified by the dispute doesn't
↪  exist you can ignore it
and assume this is an error on our partners side.

** Resolve **
A resolve represents a resolution to a dispute, releasing the associated held
↪  funds. Funds that
were previously disputed are no longer disputed. This means that the clients held
↪  funds should
```

```
decrease by the amount no longer disputed, their available funds should increase by
↪  the amount
no longer disputed, and their total funds should remain the same.
A resolve looks like
   type    client tx amount
   resolve 1      1
Like disputes, resolves do not specify an amount. Instead they refer to a
↪  transaction that was
under dispute by ID. If the tx specified doesn't exist, or the tx isn't under
↪  dispute, you can ignore
the resolve and assume this is an error on our partner's side.

** Chargeback **
A chargeback is the final state of a dispute and represents the client reversing a
↪  transaction.
Funds that were held have now been withdrawn. This means that the clients held
↪  funds and total
funds should decrease by the amount previously disputed. If a chargeback occurs the
↪  client's
account should be immediately frozen.
A chargeback looks like
   type       client tx amount
   chargeback 1      1
Like a dispute and a resolve a chargeback refers to the transaction by ID (tx) and
↪  does not
specify an amount. Like a resolve, if the tx specified doesn't exist, or the tx
↪  isn't under dispute,
you can ignore chargeback and assume this is an error on our partner's side.

IMPLEMENTATION CHOICES:
* The representation of the amount should be with u64 numbers. The decimal number
↪  is multiplied
  by 10000 basically. That should avoid any rounding error. Of course, when
  ↪  outputting value the trailing
  zero have to be cleaned up.
* Of course, any failure should lead to the operation being cancelled. The
↪  operation pass completely
  or not at all.
* A series of test must be provided so that coverage is 100%
* The code should keep in memory the account state and the transaction that can be
↪  disputed:
  + withdrawals cannot be disputed.
  + Only deposits can be disputed. If the chargeback occurs, then there is no need
  ↪  to keep it.
  + If disputed once and resolved, then it can no longer be discussed.
* Goal is to have efficient code that is still easy to read and understand.
```

Answer

Core amount and engine modules are written; now adding the CSV layer, the binary, and the tests.

…

Now the CLI integration tests with fixture files.

…

All 35 tests pass. Checking coverage, formatting, and clippy:

…

Tracking down the last 3 uncovered regions in `lib.rs` (lines are already 100%).

…

Lines and functions are at 100%; finding the 3 remaining uncovered regions precisely.

…

Coverage is 100% on regions, functions and lines, and clippy reports nothing. Last steps: a smoke run, a trailing-whitespace check, and the README.

…

I've built the payments engine in your exercise repo. All 36 tests pass (31 unit tests, 5 that run the binary), coverage is 100% for regions, functions and lines (`cargo llvm-cov`), and clippy reports no warnings. Nothing is committed yet.

How it's organised

- **`src/amount.rs`**: amounts are stored as a `u64` count of ten-thousandths, as you specified. Parsing is strict: it rejects negative numbers, more than four decimals and values too large for a `u64`. All arithmetic is checked, so an overflow fails instead of wrapping. Output drops trailing zeros (`1.5000` → `1.5`, `2.0000` → `2`).
- **`src/engine.rs`**: keeps the accounts and only the deposits that can still be disputed. Each transaction is checked completely before anything changes, so a rejected one leaves the state exactly as it was and returns a reason. `total` is calculated as `available + held` rather than stored, so the three figures can't disagree.
- **`src/lib.rs`**: reads the CSV one row at a time, reusing a single buffer, so memory depends on the number of accounts and open deposits, not on file size. Spaces around values are trimmed, columns are matched by header name, and the amount can be left out on dispute, resolve and chargeback rows.
- **`src/main.rs`**: `cargo run -- transactions.csv > accounts.csv`. It exits with code 2 if the arguments are wrong and code 1 if the file can't be read.

Choices the spec left open (also written up in `README.md`):

- A dispute, resolve or chargeback is ignored if its client doesn't match the client of the deposit it refers to.
- A dispute is rejected if the deposited money has already been withdrawn, so available funds can never go negative.
- After a chargeback, the account rejects every later transaction, including further disputes.
- A failed withdrawal from an unknown client doesn't create an empty account.
- Rows that can't be read are skipped. So is a deposit that reuses the id of a deposit still being tracked. Only a failure to read the file stops processing.

To get regions to 100%, I changed the reading and writing functions to take `&mut dyn Read` / `&mut dyn Write` instead of generic types. With generic types, the coverage tool marked 3 regions as missed even though every line ran: the tests use different reader and writer types, and the tool scores each version separately.

If you want a commit, the title would follow your style, for example `engine: Add CSV transactions engine`.

Prompt 2 2026-10-18 11:56

```
Ok, the README.md should be renamed README.
Also, document in README the design choices.
```

Answer

I renamed `README.md` to `README` with `git mv`, so git records it as a rename. I also rewrote it as plain text, since it no longer has the `.md` extension. Nothing refers to the old name and no line has trailing spaces.

The design choices section now gives the reason behind each decision:

1. **Amounts are integers:** a `u64` counting ten-thousandths, with checked arithmetic, strict parsing, and trailing zeros dropped on output.
2. **All or nothing:** each transaction is checked fully before anything changes. A rejected one leaves the state unchanged and returns a `TxError` with the reason.
3. **The total is calculated, never stored:** it is always `available + held`, so the three figures can't disagree and the total can't overflow.
4. **Only disputable deposits are kept in memory:** withdrawals aren't stored, and a deposit is dropped once its dispute is resolved or charged back.
5. **Dispute, resolve and chargeback rules:** the client must match the deposit's client, available funds can't go negative, and a deposit can't be disputed twice.
6. **A chargeback freezes the account:** every later transaction is rejected.
7. **Failures create no accounts.**
8. **Input is read one row at a time, and bad rows are skipped:** only a read or write failure stops processing.
9. **Code structure:** the engine does no reading or writing itself, and the CSV functions take `&mut dyn Read`/`&mut dyn Write` instead of generic types.

The file also covers usage, the source layout, and how to run the tests and the coverage report.

Prompt 3 2026-10-18 12:03

```
The function "fn run" in src/lib.rs:



pub fn run(input: &mut dyn Read, output: &mut dyn Write) -> Result<(), Box<dyn
↪  Error>> {
    let mut engine = Engine::new();
    process(input, &mut engine)?;
    write_accounts(&engine, output)?;
    Ok(())
}



is interesting, but I would prefer its code to be inlined in src/main.rs
We should know the state on which we are working.
```

Answer

I'll remove `run` from the library and move its steps into `main`, so the `Engine` is created and owned there. The library tests will call `process` and `write_accounts` directly.

…

The write-to-stdout error path now sits in `main` and needs a test to keep coverage at 100%. I'll add one that closes the read end of the binary's stdout pipe, so the write fails with a broken pipe.

…

Coverage is still 100% and clippy is clean. Updating the README to match:

…

I removed `run` from `src/lib.rs` and put its steps in `main`. `main` now creates the `Engine` itself and passes it to `process` (reads the CSV) and then to `write_accounts` (prints the balances), so you can see the state at each step:

```
// The whole state: client accounts and the deposits still disputable.
let mut engine = Engine::new();

let file = match File::open(path) {
    Ok(file) => file,
    Err(err) => return fail(path, err),
};
if let Err(err) = process(&mut BufReader::new(file), &mut engine) {
    return fail(path, err);
}
let mut output = BufWriter::new(io::stdout().lock());
if let Err(err) = write_accounts(&engine, &mut output) {
    return fail("standard output", err);
}
ExitCode::SUCCESS
```

Each failure is reported by a small `fail` helper, which prints `error: <context>: <err>` and exits with code 1. Errors no longer have to be converted into a common `Box<dyn Error>` type.

Other changes:

- **Library tests:** they now call `process` and `write_accounts` directly; the read-error and write-error tests work as before.
- **New test in `tests/cli.rs`:** writing to stdout can now fail inside `main`, so a new test covers it. It closes the receiving end of the binary's output, the write fails, and the test checks for exit code 1 and the `error: standard output:` message. I ran the suite three times and it passed each time.
- **`README`:** now says `main` owns the engine state, and that a failure to write the output also exits with code 1.

All 37 tests pass, clippy reports no warnings, and coverage is still 100% for regions, functions and lines.

Prompt 4 2026-10-18 12:10

```
Ok, commit the current state.

Then change the src/main.rs code so that the repeated code



    let file = match File::open(path) {
        Ok(file) => file,
```

```
        Err(err) => return fail(path, err),
    };
    if let Err(err) = process(&mut BufReader::new(file), &mut engine) {
        return fail(path, err);
    }
    let mut output = BufWriter::new(io::stdout().lock());
    if let Err(err) = write_accounts(&engine, &mut output) {
        return fail("standard output", err);
    }



is avoided. We know the pattern for that: Returning an error type.
And the use of the context can be dropped. It complexifies the
code needlessly I think and prevent the use of ? operators (which does not
have the context).
```

Answer

Now updating the CLI tests: the usage error now exits with 1, and messages start with `Error:` and no longer include the path.

…

Everything passes. Updating the README's exit-code sentence:

…

I committed the earlier state as `629db00`, with an AI-disclosure note and a `Co-Authored-By` line, as AGENTS.md asks. The `main` refactor is done but not committed.

`main` now returns a `Result` and every step uses `?`, so the repeated match blocks and the context strings are gone:

```
fn main() -> Result<(), MainError> {
    let args: Vec<String> = env::args().collect();
    let [_, path] = args.as_slice() else {
        return Err("usage: payments-engine <transactions.csv>".into());
    };

    // The whole state: client accounts and the deposits still disputable.
    let mut engine = Engine::new();

    let file = File::open(path)?;
    process(&mut BufReader::new(file), &mut engine)?;
    write_accounts(&engine, &mut BufWriter::new(io::stdout().lock()))?;
    Ok(())
}
```

Why `MainError`: when `main` returns an error, Rust prints it in debug form. For a missing file that would be `Error: Os { code: 2, kind: NotFound, message: "No such file or directory" }`. `MainError` is a small wrapper, about ten lines, that accepts any error (so `?` works) and prints the readable message instead: `Error: No such file or directory (os error 2)`.

Behaviour change: a wrong number of arguments now exits with code 1 instead of 2, since every error goes through the same path. If you want code 2 back, I can add a `process::exit(2)` for that one case.

Other updates:

- **Tests:** the CLI tests now share an `assert_error` helper that checks for exit code 1, a message

starting with `Error:` containing the expected text, and empty stdout.
- **`README`:** the exit-code sentence is updated.

All 37 tests pass, clippy reports no warnings, and coverage is still 100% for regions, functions and lines. If you want this committed, a fitting title would be `main: Report errors through a returned error type`.

Prompt 5 2026-10-18 12:21

```
Good point.
Now is there a nice rust way to handle that?
The no error is typically 0. Could we pass the error kind in MainError?
Or is making the code coherent with the return code requiring complex handling?
And note that 2 is not great. We want the POSIX error codes that are defined in
↪  errno.h
Not explicit magic constants.
```

Answer

Now the CLI tests check the exact errno for each failure:

…

csv's `From<csv::Error> for io::Error` wraps the whole error and drops the OS code, so it doesn't work here. I'll read the OS code from `csv::ErrorKind::Io` directly instead.

…

All green at 100%. Updating the README's exit-status sentence:

…

Yes, and it's simple. The program now exits with the error's errno.h code: `ENOENT` (2) for a missing file, `EISDIR` (21) for a directory, `EPIPE` when stdout is closed, `EINVAL` for wrong arguments, and `EIO` when an error has no OS code. All 42 tests pass, clippy reports no warnings, and coverage is still 100%. This change is not committed.

Your questions:

- **Can `MainError` carry the code?** Yes, it now holds the exit code and the message. But a `main` that returns `Result` always exits 1. That rule is built into the standard library, we can't change it, and the way around it isn't in stable Rust yet. So `main` returns an `ExitCode` instead, and the code using `?` moves to `try_main`, in `main.rs`, where the `Engine` is still created:

  ```
  fn main() -> ExitCode {
      match try_main() {
          Ok(()) => ExitCode::SUCCESS,
          Err(err) => {
              eprintln!("error: {}", err.message);
              ExitCode::from(err.code)
          }
      }
  }
  ```

  That four-line match is the only extra handling; `try_main` is the same `?` chain as before.
- **No magic constants:** a system error already carries its errno.h code (`io::Error::raw_os_error()`). The only codes I chose are `libc::EINVAL` for usage and `libc::EIO` as the fallback, both named constants from the new `libc` dependency.

Details worth knowing:

- **CSV read errors:** the obvious conversion through `io::Error` loses the system code (the directory test failed on it). So I read the code from the CSV library's I/O error variant directly.
- **Codes that don't fit:** an exit code is one byte, so `exit_status` turns any value outside 1–255 into `EIO`. In particular, 0 can never be reported as success.
- **Tests:** the CLI tests now check the exact errno for each failure. New unit tests in `main.rs` cover the conversions and fallbacks.
- **README:** it now describes the exit codes.

Using errno values as exit codes isn't a common convention. Shells reserve 126 and above, and BSD's `sysexits.h` (`EX_USAGE`, `EX_NOINPUT`, `EX_IOERR`) is the usual choice. errno codes never collide with the shell's, so this is safe as long as anything that reads the exit code knows the convention.

Prompt 6 2026-10-18 12:29

```
Ok, commit it.

Add to the README that a frozen account cannot be unfrozen. Some significant fact
↪  about the specification.
```

Answer

Committed as `5d0cf18`. Now the README addition:

…

I committed the errno change as `5d0cf18`, then added the frozen-account note to the README. That README change is not committed.

It goes under design choice 6, "A chargeback freezes the account":

> A frozen account cannot be unfrozen: the specification defines no transaction that unlocks an account, so the lock is permanent. As a consequence, funds of other deposits still under dispute when the account is frozen stay held forever, since their resolve or chargeback is rejected too.

The second sentence is the less obvious consequence. Because a locked account rejects everything, any other dispute that was open at the time of the chargeback can never be settled, so its money stays held. The engine test `locked_account_rejects_everything` checks exactly this: 4 units stay held after the lock.

Should I commit it, as something like `README: Note that a frozen account cannot be unfrozen`?

Prompt 7 2026-10-18 12:50

```
Ok, commit it.

Actually, we have to rethink the design.
If someone does a deposit of 10, a withdraw of 10 and then the dispute of the
↪  deposit of 10.
In the code that we have the dispute is not possible.

But we want that when this happens the available fund becomes negative. The user is
↪  now
in debt and something would happen though it is out of scope of this work.
```

```
So, Amount has to allow for negative values. So, it should be "i64".

That being said, if the available founds are negative, then further withdrawal
↪  should
be impossible.
```

Answer

Now the engine:

…

Tests pass. Next, an end-to-end CSV test of your exact scenario, to check that negative values are printed correctly:

…

Coverage is still 100%. Now the README: choices 1 and 5, and the frozen-account note, which mentioned "held forever".

…

I committed the README note as `bf156f4`, then made the redesign: your scenario (deposit 10, withdraw 10, dispute the deposit) is now accepted and leaves the client in debt. This redesign is not committed yet. All 50 tests pass, clippy reports no warnings, and coverage is still 100% for regions, functions and lines.

Rules now:

- **Disputes:** always accepted, even if the money was already withdrawn. Your scenario gives `available -10, held 10, total 0`.
- **Withdrawals:** accepted only if `available` stays at zero or above, so a client in debt can't withdraw anything, not even 0. Deposits are still accepted and reduce the debt.
- **Resolve:** returns the held money to available and clears the debt (`10 / 0 / 10` in your scenario).
- **Chargeback:** the debt stays: `available -10, held 0, total -10`, locked.

Changes in the code:

- **`Amount` uses `i64`.** Negative values are printed correctly (`-0.5`, and even `i64::MIN`). Amounts read from the CSV still can't be negative, so `-1` in the input is still rejected.
- **The overflow check on disputes is now real.** Before, it could never fail. Now a dispute can push `available` below the `i64` minimum, or push `held` above the maximum while `available` still fits. A test for each builds that exact state. Both values are computed before anything changes, so a failed dispute still changes nothing.
- **Tests:** the old test expecting a rejected dispute became a debt test. New tests cover being in debt, resolve versus chargeback, and the two overflow cases, plus a CSV-level test where a later withdrawal of 0.0001 fails.
- **`README`:** the parts on signed amounts, the derived total, and the dispute/debt rules (with your example) are updated.

One more consequence: `total` can now be negative after a chargeback.

Should I commit it, as something like `engine: Allow disputes to put a client in debt`?

Prompt 8 2026-10-18 12:55

```
Yes, commit it.
```

Answer

I committed the redesign as `607698b` ("engine: Allow disputes to put a client in debt"), with the AI-disclosure note and `Co-Authored-By` line. It includes the signed amounts, the debt and withdrawal rules, the new tests and the README update. The working tree is clean and there are now five commits on `main`. Nothing has been pushed.